

Języki Skryptowe

Dokumentacja projektu własnego - kalendarz

Dawid Pacia, grupa 4/8

22.01/2024

1. Opis programu

Projekt własny – Kalendarz.

Program służy jako kalendarz którego użyć można do śledzenia i planowania zadań dla poszczególnych dni. Pokazuje na start obecny miesiąc. Można przemieszczać się do następnych i poprzednich miesięcy i także dla nich tworzyć zadania, które wpisywać można klikając na poszczególne dni

2. Instrukcja obsługi

Należy uruchomić plik start.bat który wyświetli nam menu. **Opcja pierwsza** uruchomi program i po zakończeniu naszej pracy uruchomi plik raport.py który wygeneruje raport.html.

Aby zapisać dane wydarzenie dla danego dnia należy kliknąć w dany dzień i wpisać w pole tekstowe tekst, który chcemy tam umieścić. Następnie po zamknięciu okna tekstowego przyciskiem X w prawym górnym rogu, gdy chcemy aby dane zostały zapisane do pliku należy nacisnąć przycisk Save. Przycisk save zapisze wszystkie wprowadzone zmiany do pliku. Po wyjściu zostanie utworzony raport z listą aktualnie znajdujących się w pliku pozycji.

W raporcie znajdzie się także data w której dany raport został wygenerowany.

Opcja druga utworzy folder Backup jeśli go nie ma i zapisze do niego obecnie znajdujący się w folderze raport.html oraz plik z danymi: data.txt. Należy się upewnić że pliki takie się tam znajdują czyli najpierw użyć opcji 1, aby pliki wygenerować. Gdy tego nie zrobimy zostanie nam wyświetlony odpowiedni komunikat który powie nam o tym że tych plików brakuje.

Opcja trzecia zamyka terminal.

Dodatkowe informacje

Wymagania:

- python 3.12 (dla wcześniejszych wersji mogą wystąpić problemy)
- tkinter 8.6
- customtkinter 5.2.1

3. Opis działania

Dane przechowywane są w pliku dane.txt. Program na początku wczytuje dane jeśli plik istnieje lub tworzy go gdy nie istnieje. Program tworzy następnie interfejs i ustawia hierarchie elementów i je konfiguruje.

Następnie wykonywana jest pętla w której sprawdzane jest czy nie zostały naciśnięte któreś z interaktywnych elementów interfejsu. Jeśli zostały naciśnięte wykonywana jest przypisana do elementu funkcja.

Gdy otworzymy za pomocą start.bat wyświetlą nam się 3 opcje:

```
#####  
#                               #  
##### MENU #####  
#####  
1. Uruchom program i utwórz raport #  
2. Utwórz kopie zapasowe plików do folderu Backup #  
3. Wyjście #  
#####  
Wybierz(1 / 2 / 3):
```

Pierwsza opcja otwiera aplikację a po jej zamknięciu utworzy raport czytując z pliku data.txt wszystkie pozycje i je zapisze. Na górze będzie także widniała data kiedy raport został utworzony.



```
C:\Windows\system32\cmd.exe

#####
#                               MENU                               #
#####
1. Uruchom program i utwórz raport                                #
2. Utwórz kopie zapasowe plików do folderu Backup                #
3. Wyjście                                                         #
#####
Wybierz(1 / 2 / 3): 1
Uruchamiam aplikację...
Tworzę raport...
Press any key to continue . . .
```

Raport z aplikacji kalendarz

Wygenerowano w dniu: 2024-01-23

- 01-01-2024
zrobić zadania
odebrać przesyłkę
- 02-01-2024
kolokwium z języków skryptowych
- 03-01-2024
ograniczyć programowania
projekt z javy
- 04-01-2024
dokończyć projekt z języków skryptowych
- 12-01-2024
przygotować się do prezentacji

Druga opcja utworzy kopie plików(jeśli istnieją) data.txt oraz raport.html i zapisze do folderu Backup. Jeśli folder nie istnieje to najpierw go utworzy.

```
C:\Windows\system32\cmd.exe

#####
#                               #
#####
1. Uruchom program i utworz raport      #
2. Utworz kopie zapasowa plikow do folderu Backup  #
3. Wyjście                               #
#####
Wybierz(1 / 2 / 3): 2
Kopiuje plik data.txt...
C:data.txt
1 File(s) copied
Kopiuje plik raport.html...
C:raport.html
1 File(s) copied
Proces kopiowania zakonczony.
Press any key to continue . . .
```

```
C:\Windows\system32\cmd.exe

#####
#                               #
#####
1. Uruchom program i utworz raport      #
2. Utworz kopie zapasowa plikow do folderu Backup  #
3. Wyjście                               #
#####
Wybierz(1 / 2 / 3): 2
plik data.txt nie istnieje utworz go zanim spróbujesz zrobic kopie
plik raport.html nie istnieje utworz go zanim spróbujesz zrobic kopie
Proces kopiowania zakonczony.
Press any key to continue . . .
```

Trzecia opcja zakończy pracę i zamknie konsolę

Test na wybranie opcji która nie istnieje:

```
#####
#                               #
#####
1. Uruchom program i utworz raport      #
2. Utworz kopie zapasowa plikow do folderu Backup  #
3. Wyjście                               #
#####
Wybierz(1 / 2 / 3): asdfg
Taka opcja nie istnieje...
Wróć do menu...
Press any key to continue . . .
```

4. Implementacja

Cała implementacja aplikacji znajduje się w pliku calendarapp.py oraz dane przechowywane są w pliku data.txt.

Kod używa kilku globalnych zmiennych current_year oraz current_month do których na początku są wczytywane wartości dnia obecnego a potem zmienne są modyfikowane przy poruszaniu się po kalendarzu.

Użyte biblioteki:

- tkinter – elementy interfejsu
- customtkinter – elementy interfejsu
- calendar – sprawdzenie ilości dni w wybranym miesiącu i zamiana numeru na nazwę miesiąca
- math - obliczenia
- datetime – wczytanie obecnej daty do globalnych zmiennych
- os – sprawdzanie istnienia pliku

Dane w pliku data.txt zapisywane są w następujący sposób

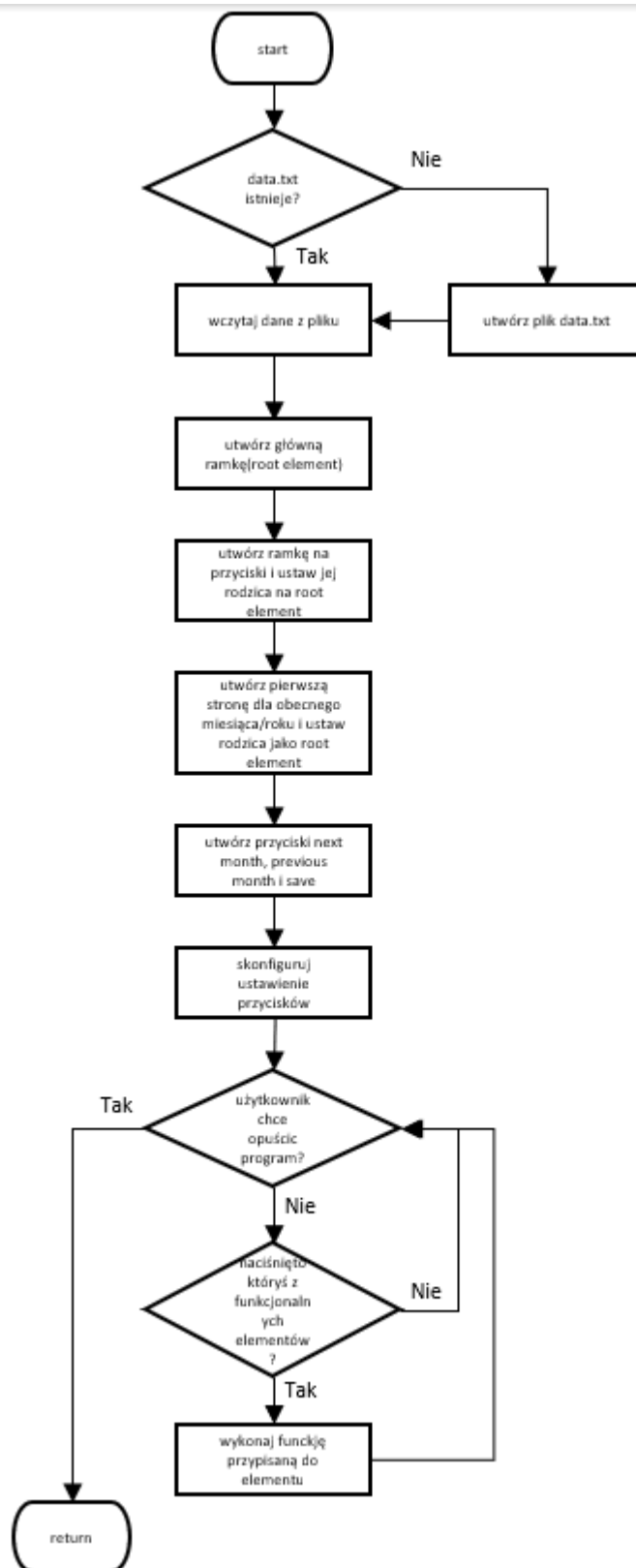
```
#01-01-2024
zrobic zadania
odebrac przesylke
#02-01-2024
kolokwium z jezykow skryptowych
#03-01-2024
egzamin z programowania
projekt z javy
#04-01-2024
dokonczyć projekt z jezykow skryptowych
#12-01-2024
przygotować się do prezentacji
```

Końcowa struktura folderu:

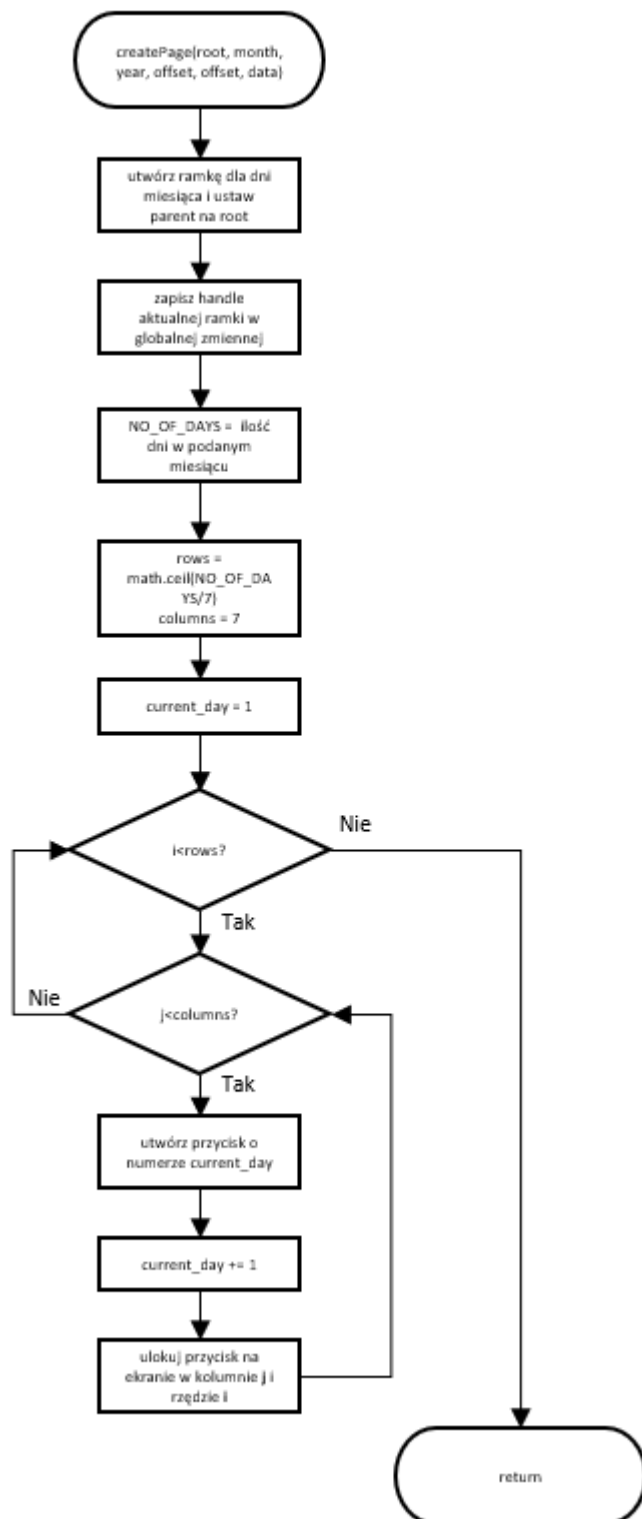
```
C:\USERS\ DARKNESSO\DESKTOP\KALENDARZ
calendarapp.py
data.txt
raport.html
raport.py
start.bat

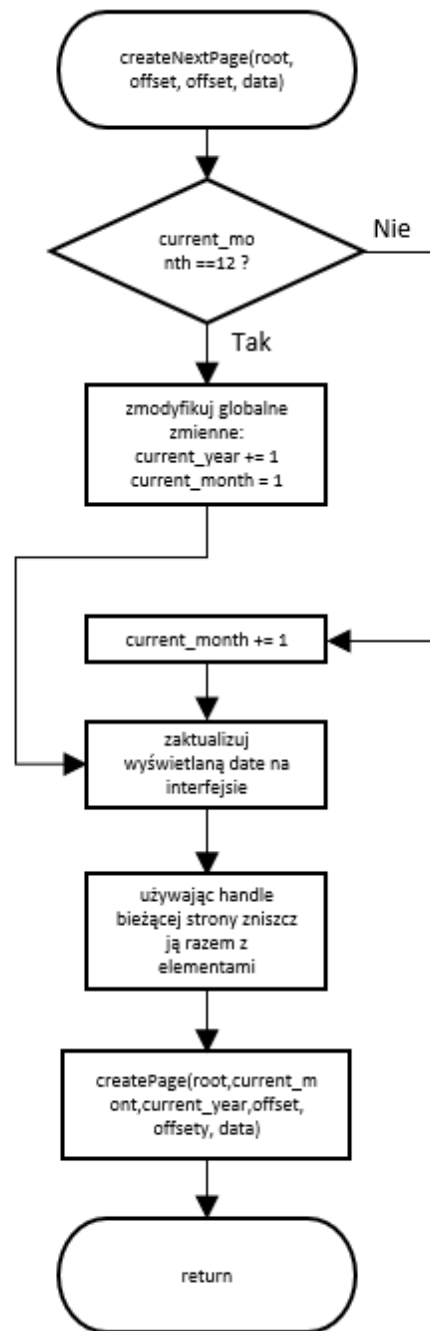
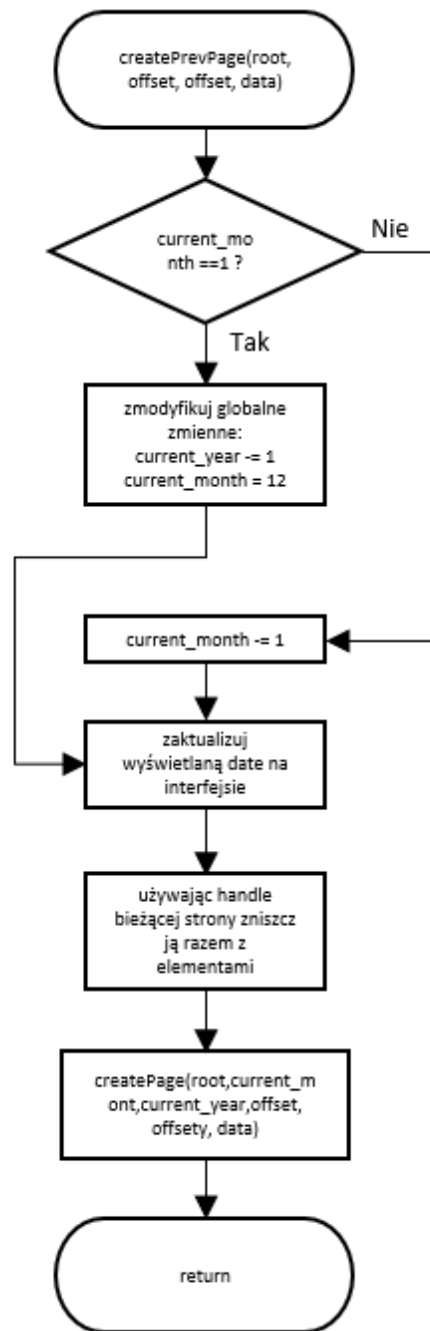
Backup
data.txt
raport.html
```

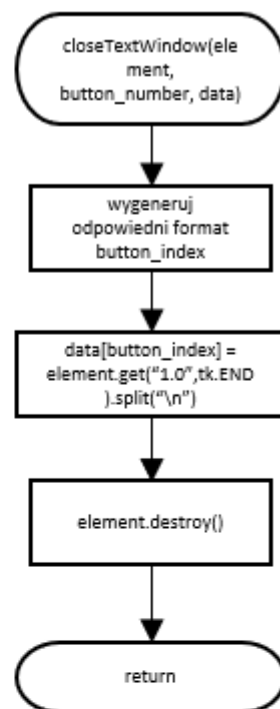
Schemat blokowy głównego kodu:

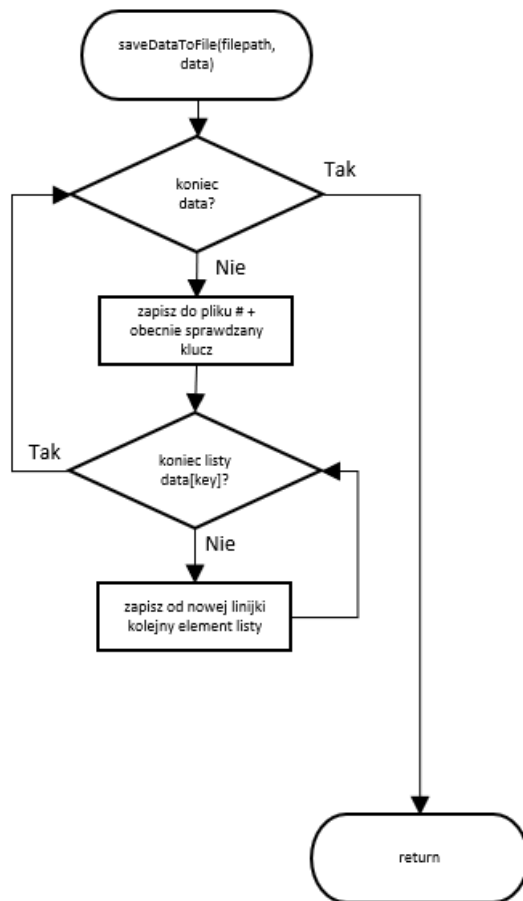
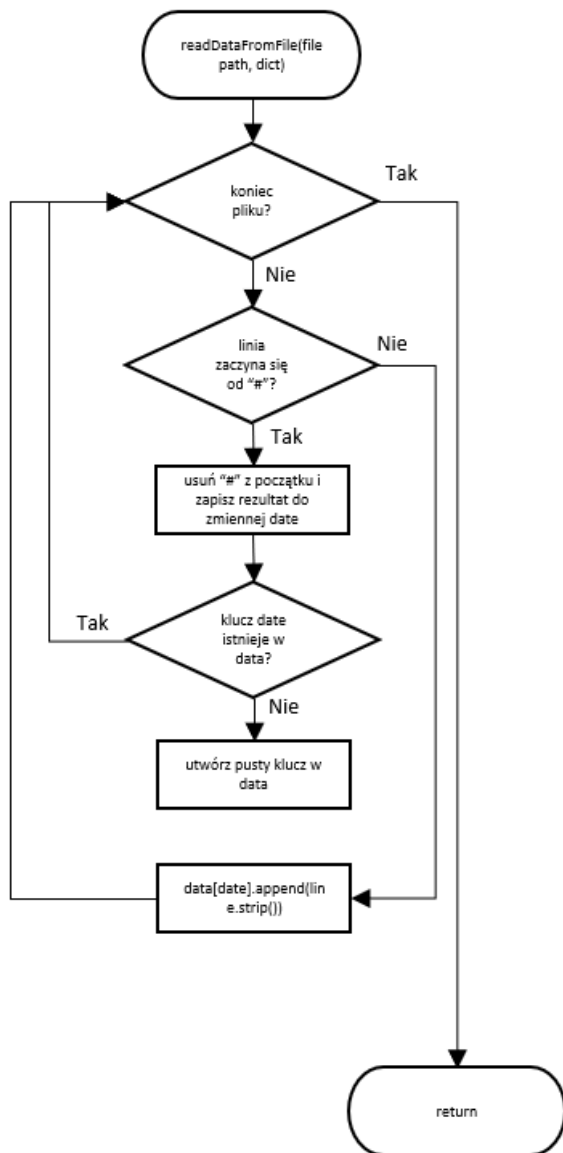


Schematy blokowe funkcji:









Kod calendarapp.py:

```
import tkinter as tk

import customtkinter as ctk

import calendar as cal

import math

from datetime import date

import os


current_month = date.today().month

current_year = date.today().year

daysData = {}


def generate_dict_index(button_number):

    if (button_number > 0 and button_number < 10):

        if(current_month < 10):

            button_index = f"0{button_number}-0{current_month}-{current_year}"

        else:

            button_index = f"0{button_number}-{current_month}-{current_year}"

    else:

        if(current_month < 10):

            button_index = f"{button_number}-0{current_month}-{current_year}"

        else:

            button_index = f"{button_number}-{current_month}-{current_year}"

    return button_index


def saveDataToFile(filepath, data):

    with open(filepath, "w") as myFile:

        for key in data:

            write = False

            for element in data[key]:

                if element.strip() != "" and element != "":

                    write = True

                    break;

            if write==True:

                myFile.write(f"#{key}\n")

            for element in data[key]:
```

```

        if element.strip() != "" and element != "":
            myFile.write(f"{element}\n")

def readDataFromFile(filepath, dict):
    with open(filepath,"r") as myFile:
        for line in myFile:
            if line[0] == "#":
                date = line.removeprefix("#").strip()
                day = int(date[0:2:1])
                if (day > 0 and day < 32 and date not in dict):
                    dict[date] = []

            elif (date in dict):
                if line.strip() != "":
                    dict[date].append(line.strip())

def closeTextWindow(element,button_number,data):
    button_index = generate_dict_index(button_number)
    data[button_index] = element.get("1.0",tk.END).split("\n")
    element.destroy()

def openTaskList(root,button_number, data):
    button_index = generate_dict_index(button_number)
    tasks = ""
    if button_index in data:
        for task in data[button_index]:
            tasks += task + '\n'
    else:
        data[button_index] = {}
    textBox = ctk.CTkTextbox(root, width = 500, height = 500, bg_color="#476c9b")
    textBox.insert(tk.END, tasks)
    textBox.place(x = 256, y = 100)
    closeButton = ctk.CTkButton(textBox, width = 10, height = 10, text = "X", text_color="black", corner_radius=20
,fg_color="#981722", bg_color="#1d1e1e", command = lambda: closeTextWindow(textBox, button_number, data))
    closeButton.place(relx = 0.96, rely= 0.0)

def createPage(root, month, year , offsetx, offsety, data):
    dayFrame = ctk.CTkFrame(root, width = 1009, height = 725, fg_color="#94BEDE")

```

```

dayFrame.pack()

global currentFrameHandle

currentFrameHandle = dayFrame

NO_OF_DAYS = cal.monthrange(year, month)[1]

current_day = 1

rows = math.ceil(NO_OF_DAYS/7)

columns = 7

saved_offsetx = offsetx

for i in range(rows):

    for j in range(columns):

        if(current_day > NO_OF_DAYS):

            break

        else:

            button = ctk.CTkButton(dayFrame,

                width=140,

                height=140,

                fg_color="#476C9B",

                hover_color="#41618B",

                border_color="#101419",

                border_width=2,

                text = 7*i+j+1,

                text_color="#323639",

                corner_radius=20,

                font = ("Roboto Mono",30),

                command = lambda i = current_day: openTaskList(root, i, data)).grid(column = j, row = i, padx = 2, pady = 2)

            offsetx += 143

            current_day += 1

        offsetx = saved_offsetx

        offsety += 143

def createNextPage(root, offsetx, offsety, data):

    global current_year

    global current_month

    global currentFrameHandle

    global dateLabel

    if current_month == 12:

        current_year += 1

```

```

        current_month = 1

    else:

        current_month += 1

    dateLabel.configure(text=f"{cal.month_name[current_month]} / {current_year}")

    currentFrameHandle.destroy()

    createPage(root, current_month, current_year,offsetx, offsety, data)


def createPrevPage(root, offsetx, offsety,data):

    global current_year

    global current_month

    global currentFrameHandle

    global dateLabel

    if current_month == 1:

        current_year -= 1

        current_month = 12

    else:

        current_month += -1

    dateLabel.configure(text=f"{cal.month_name[current_month]} / {current_year}")

    currentFrameHandle.destroy()

    createPage(root, current_month, current_year,offsetx, offsety, data)


# if config file not present create an empty one

if not os.path.exists("data.txt"):

    with open("data.txt","w") as file:

        file.write("")


# read data for the first time and save it in a dictionary

readDataFromFile("data.txt", daysData)


# creating root element

window = ctk.CTk(fg_color="#94BEDE")

window.geometry('1009x765')

window.title("Calendar")

window.resizable(width = False, height = False)


# top button frame creation

buttonFrame = ctk.CTkFrame(window,width = 1000, height=50, fg_color="#94BEDE")

buttonFrame.pack(fill=tk.BOTH)

```

```

# create first page

createPage(window, current_month, current_year, 73, 110, daysData)


# button creation

dateLabel = ctk.CTkLabel(buttonFrame, width = 150 ,text=f"{cal.month_name[current_month]} /
{current_year}",corner_radius=7, fg_color="#1E2A3A", text_color="#FAFAFF")

nextButton = ctk.CTkButton(buttonFrame, text = "Next month", fg_color="#1E2A3A", hover_color="#2D3D4F",
text_color="#FAFAFF", font = ("Roboto Mono",15), corner_radius=20, command = lambda: createNextPage(window,73,
110,daysData))

prevButton = ctk.CTkButton(buttonFrame, text = "Previous month", fg_color="#1E2A3A", text_color="#FAFAFF", font =
("Roboto Mono",15), corner_radius=20, command = lambda: createPrevPage(window,73, 110,daysData))

saveButton = ctk.CTkButton(buttonFrame, text = "Save", fg_color="#1E2A3A", hover_color="#2D3D4F", text_color="#FAFAFF",
font = ("Roboto Mono",15), command = lambda:saveDataToFile("data.txt",daysData))


# configuring the button layout

dateLabel.grid(column=1,row=1, padx = 67)

prevButton.grid(column=3,row=1,padx = 3, pady = 7 )

saveButton.grid(column=4,row=1,padx = 3, pady = 7)

nextButton.grid(column=5,row=1,padx = 3, pady = 7)

window.mainloop()

```

Kod raport.py

```

from datetime import date

```

```

with open("raport.html", "w") as raport:

```

```

    raport.write("<html>\n"

        "<head>\n"

        "<style type=\"text/css\">\n"

        "h1 {\n"

        "margin-bottom: 1px;\n"

        "}\n"

        "</style>\n"

        "<title>Raport</title>\n"

        "</head>\n"

        "<h1>Raport z aplikacji kalendarz</h1>\n"

        f"<p><font size=\"4\"> Wygenerowano w dniu: {date.today()} </font></p>\n"

        "<div style=\"white-space: pre\">\n")

```

```

with open("data.txt", "r") as data:

```

```

for line in data:

    if line[0] == "#":

        raport.write('\n')

        raport.write(line.removeprefix("#").strip())

        raport.write('\n')

    else:

        raport.write(line)

raport.write("</div>\n"

            "</body>\n"

            "</html>\n")

```

Kod start.bat:

```

@echo off

setlocal enabledelayedexpansion

:menu

cls

echo #####

echo #          MENU          #

echo #####

echo 1. Uruchom program i utworz raport          #

echo 2. Utworz kopie zapasowa plikow do folderu Backup #

echo 3. Wyjscie          #

echo #####

set /p choice=Wybierz(1 / 2 / 3):

if %choice%==1 (

    goto ch1

) else if %choice%==2 (

    goto ch2

) else if %choice%==3 (

    goto :EOF

) else (

    goto ch3

)

:ch1

```


echo Uruchamiam aplikacje...

python -u calendarapp.py

echo Tworze raport...

python raport.py

pause

goto menu

:ch2

if not exist Backup\ (

mkdir Backup\

)

if exist "data.txt" (

echo Kopiuje plik data.txt...

xcopy data.txt Backup\

)

if exist "raport.html" (

echo Kopiuje plik raport.html...

xcopy raport.html Backup\

)

if not exist "data.txt" (

echo plik data.txt nie istnieje utworz go zanim spróbujesz zrobic kopie

)

if not exist "raport.html" (

echo plik raport.html nie istnieje utworz go zanim spróbujesz zrobic kopie

)

echo Proces kopiowania zakonczony.

pause

goto menu

:ch3

echo Taka opcja nie istnieje...

echo Wracanie do menu...

pause

goto menu